Multi-Dimensional Arrays & Build Automation

CSC 230 : C and Software Tools

NC State Department of Computer Science

Topics for Today

- Multi-Dimensional Arrays
- Program Execution
- Build automation with Make

Multi-Dimensional Arrays

- To make a multi-dimensional array, you stack up square brackets after the variable name.
- Declared like this, you have to give a size for all dimensions.

```
int table[ 3 ][ 4 ];
```

Outer dimension
(normally we'd think of this as rows)

Inner dimension
(normally we'd think of this as columns)

- Accessing an element, just like you'd expect,
  - an index for row, then column, inside square brackets.

```
x = table[ 2 ][ 3 ];
table[ 1 ][ 0 ] = 99;
```

Multi-Dimensional Array Initialization

- With initialization, elements are initialized
  - row-by row, each row in curly brackets
  - and, left-to-right within each row.

0	1	4	9	16
25	36	49	64	81
100	121	144	169	196

```
int table[ 3 ][ 5 ] = {
  {0, 1, 4, 9, 16},
  {25, 36, 49, 64, 81},
  {100, 121, 144, 169, 196}
};

for( int i = 0; i < 3; i++ )
    for( int j = 0; j < 5; j++ )
        printf( "%d ", table[i][j] );
```

I'm the first row.

I'm the first element on the first row.

Prints 0 1 4 9 16 25 …

Partial Initialization

- You can do partial initialization in a 2D array

```
int table[3][5] = {
   { 0, 1 },
   { 25, 36, 49, 64, 81 },
   { 100 }
};
```

You're going to get some zeros.

0	1	0	0	0
25	36	49	64	81
100	0	0	0	0

- Or, with C99 syntax

```
int table[3][5] = {
   [ 1 ] = { 25, 36, 49, 81 },
   { [ 4 ] = 196 },
   [ 0 ][ 2 ] = 4
};
```

0	0	4	0	0
25	36	49	64	0
0	0	0	0	196

Multi-Dimensional Arrays

- Multi-dimensional arrays are just really **arrays of arrays**.

```
int row[ 5 ];

int table[ 3 ][ 5 ];
```

I'm an array of 5 ints.

I'm an array of 3 guys like row.

- We can take an array apart, into its individual elements (rows)
  - .. and each row's elements (ints)

```
sizeof( table );
sizeof( table[ 0 ] );
sizeof( table[ 0 ][ 0 ] );
```

I'm the size of the whole array.

I'm the size of one of its elements, a row.

I'm the size of one of that row's elements.

Multi-Dimensional Arrays

- Multi-dimensional arrays are just really **arrays of arrays**.

```
int row[ 5 ];

int table[ 3 ][ 5 ];
```

- We can take an array apart, into its individual elements (rows)
  - .. and each row's elements (ints)

```
sizeof( table );
sizeof( table[ 0 ] );
sizeof( table[ 0 ][ 0 ] );
```

3 * 5 * 4 = 60 bytes

5 * 4 = 20 bytes

4 bytes

Arrays of Strings

- You can use a 2D char array to store a list of strings.

```
char words[ 20 ][ 30 ] = {
   "apple",
   "ball"
};
```

I'm like a list of 20 strings, each up to 29 characterslong.

String initialization syntax even works on the rows.

- Each row is just like any other string.

```
scanf( "%29s", words[ i ] );
printf( "%s", words[ j ] );
int len = strlen( words[ k ] );
```

Look at me! I'm a string!

Multi-Dimensional Array Initialization

- You can let the compiler determine the array size
  - But just the outermost (first) dimension.

```
int a[ ][ 3 ] = { { 10, 15, 20 },
                  { 30, 35, 40 } };
```

OK

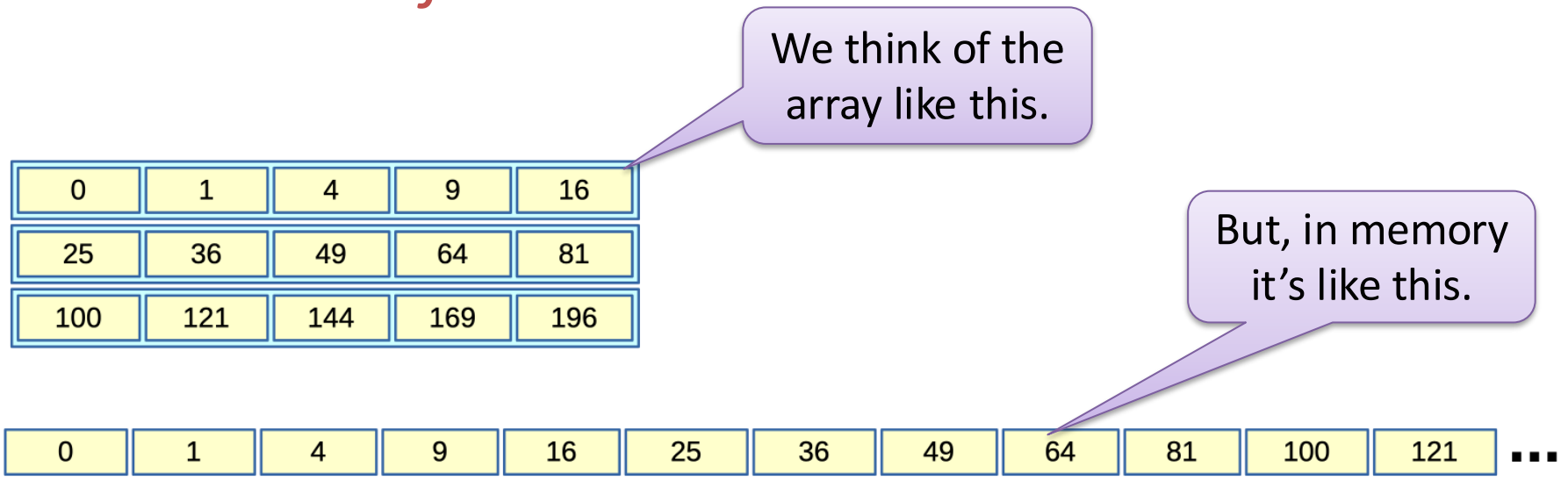
```
int a[ 2 ][ ] = { { 10, 15, 20 },
                  { 30, 35, 40 } };
```

Not OK

- Why the difference?
  - Size of all inner dimensions must be specified in the declaration.
  - It's used by the compiler to write the indexing code.

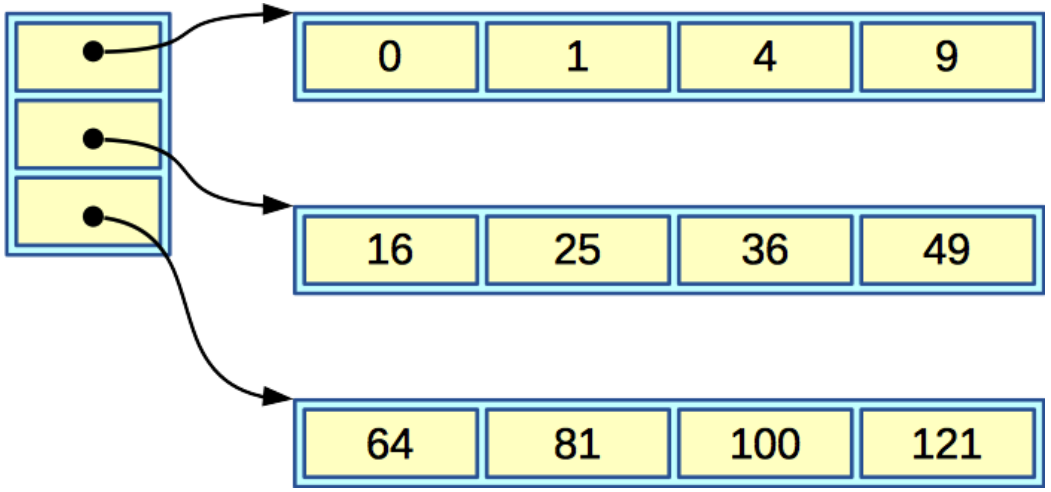
Memory Organization

- In memory, two-dimensional arrays are stored in *row-major order*



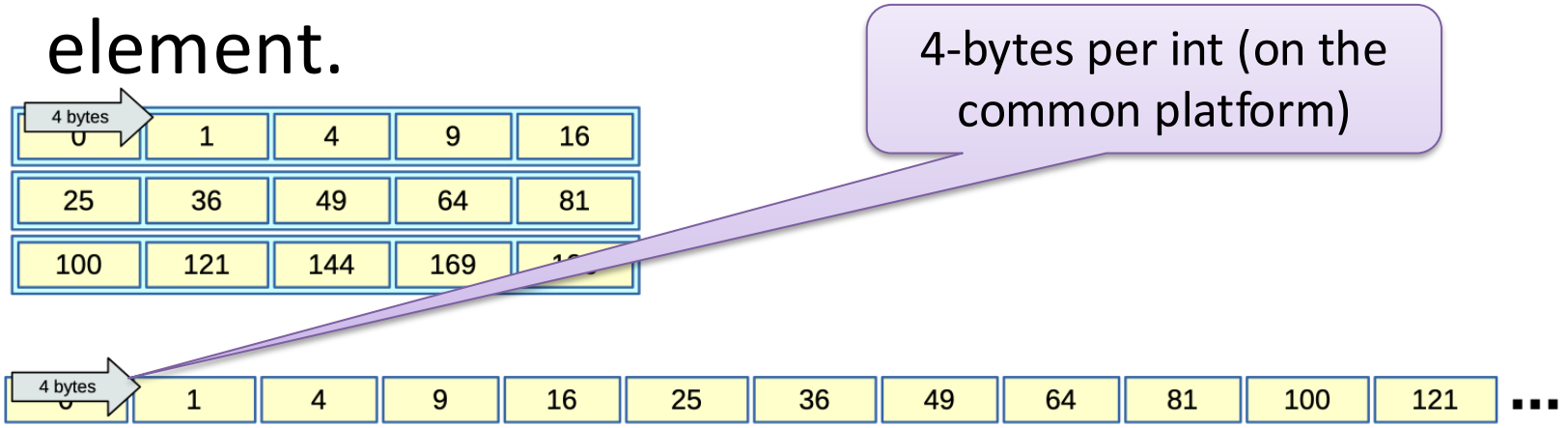
Memory Organization

- It's not like this in Java.
  - It creates multi-dimensional arrays more like:

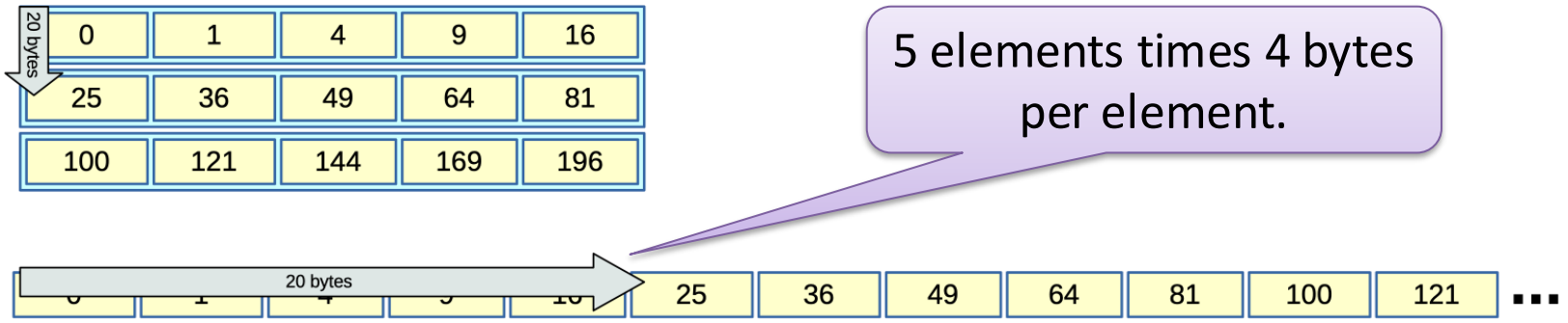


2D Array Indexing

- Moving one column, that's the size of an element.

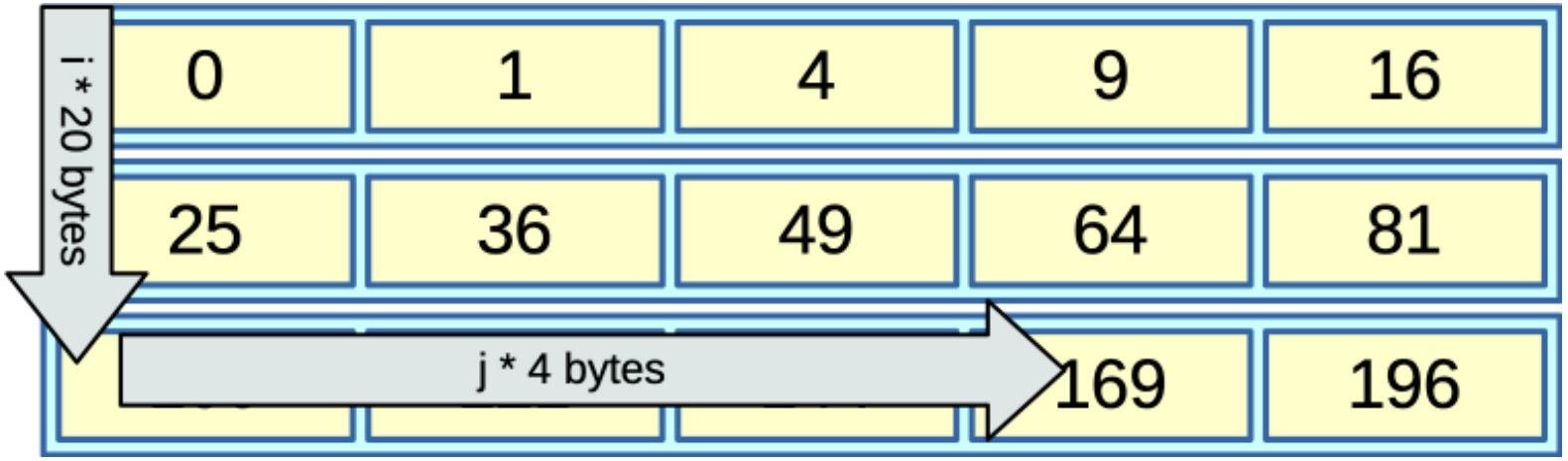


- Moving one row equals the size of the row.



Array Indexing

- Looking up a[i][j]




```
a +
i * columns * sizeof( element ) +
j * sizeof( element )
```

Array Dimensions

To use the array, I need to know the number of columns.

… strange, no mention of the number of rows.

```
a +
i * columns * sizeof( element ) +
j * sizeof( element )
```

I'm willing to make inferences about the number of rows.

… but you will need to be clear about the number of columns.

```
int a[ ][ 3 ] = { { 10, 15, 20 },
                  { 30, 35, 40 } };
```

Array Parameter Dimensions

- In array passing, the compiler doesn't care so much about the outermost (first) dimension.
- But it cares a lot about the inner sizes.

```
void report( int table[5][8] )
{
    …
}
```

```
int regularTable[5][8];
int    tallTable[6][8];
int    wideTable[5][9];
…
report( regularTable );
report( tallTable );
report( wideTable );
```

This will work

So will this.

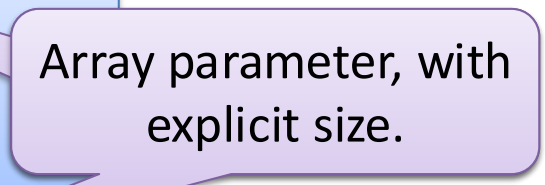
But not this!

Array Parameter Dimensions

- The array parameter looks like an array declaration
- You can specify the array length explicitly in the function declaration … if you want.

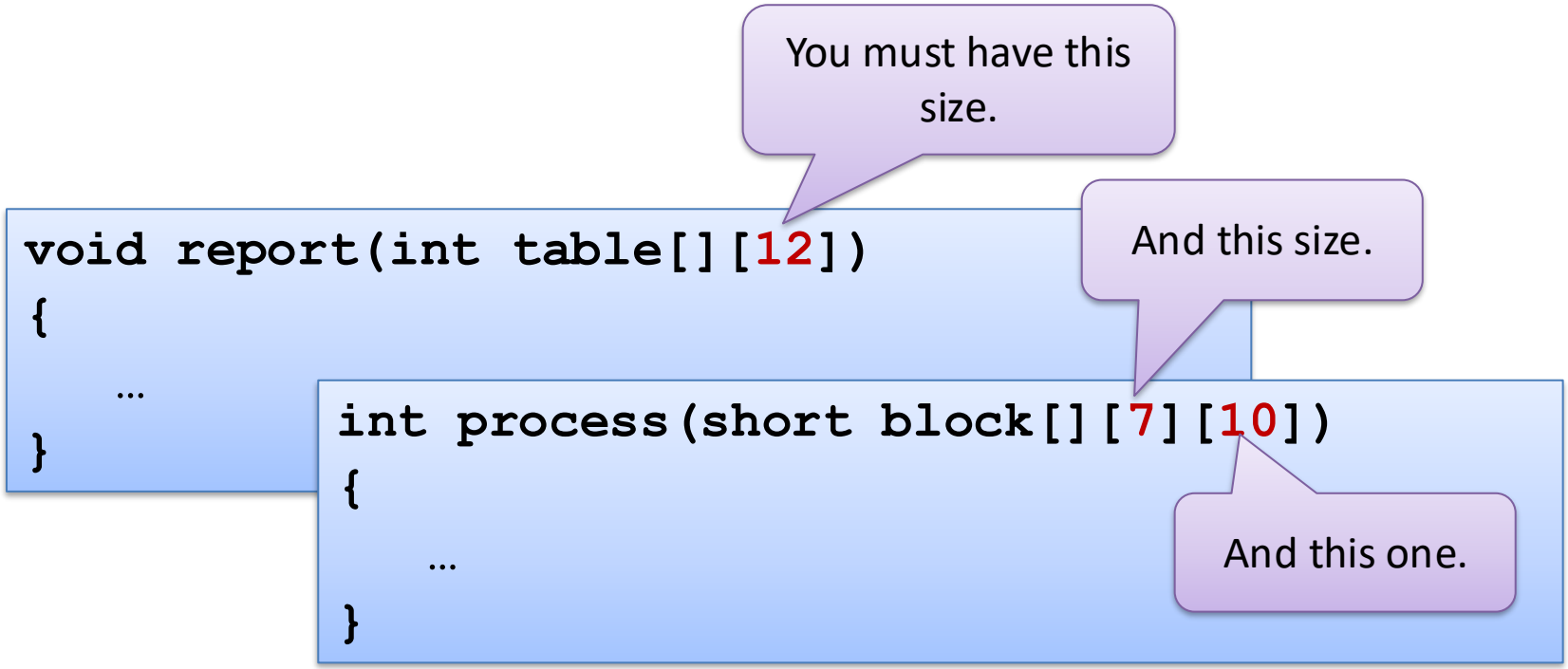
```
void report( int table[8] )
{
    …
}
```

```
void report( int table[5][8] )
{
    …
}
```



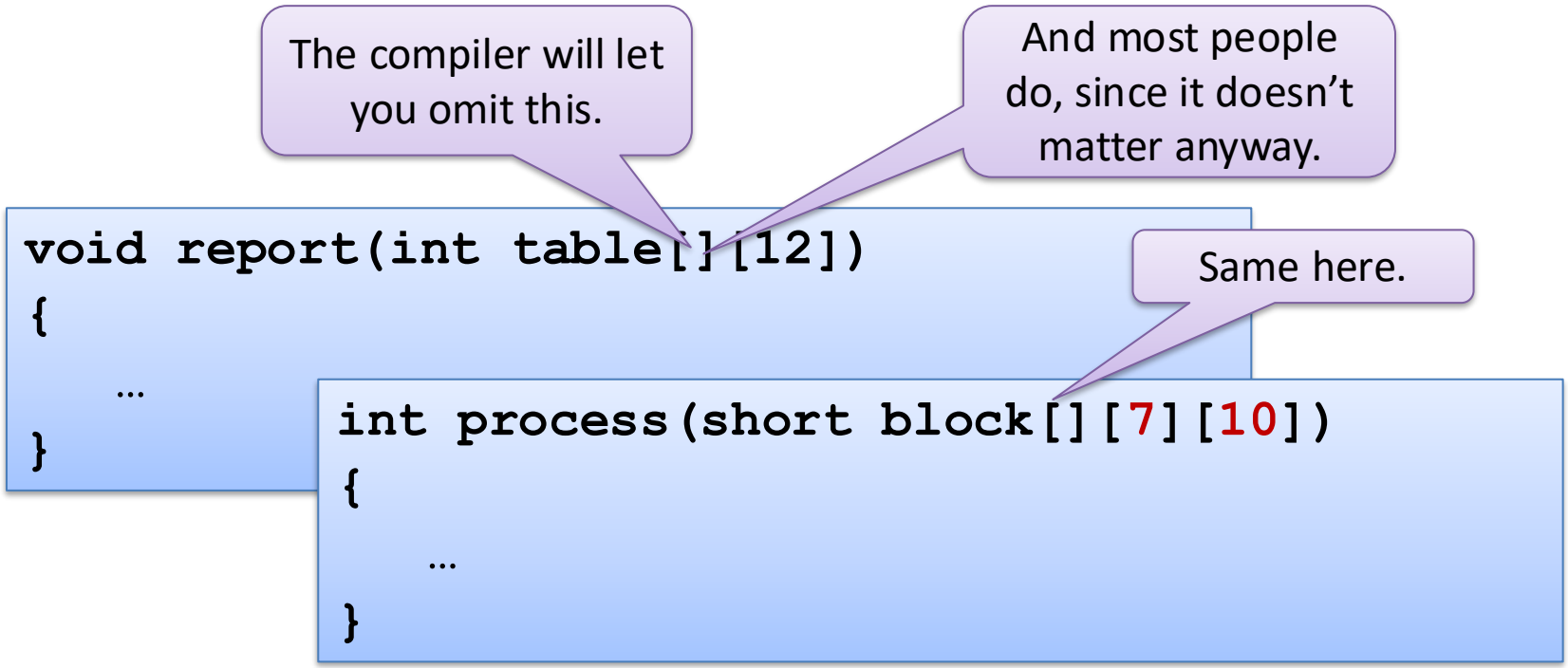
Array Parameter Dimensions

- In array indexing, the inner size determines the width of each row.
  - The compiler needs to know this to index between the rows..
- But, the outermost index just determines how far we go before we're out of bounds
  - And C doesn't do any bounds checking anyway.



Array Parameter Dimensions

- In array indexing, the inner size determines the width of each row.
  - The compiler needs to know this to index between the rows..
- But, the outermost index just determines how far we go before we're out of bounds
  - And C doesn't do any bounds checking anyway.



Passing Variable-Sized Arrays

- For variable-sized arrays, you **must** pass everything but the outer-most dimension.

You need this just to pass the array's type.

See, it gets used here in the type.

```
void sumTable(int rows, int cols, int table[][cols])
{   …
    for ( i = 0 ; i < rows; i++)
        for ( j = 0; j < cols; j++)
            total += table[i][j];
    …
}
```

Passing Variable-Sized Arrays

- For variable-sized arrays, you **must** pass everything but the outer-most dimension.

Outermost dimension isn't required for the array's type.

```
void sumTable(int rows, int cols, int table[][cols])
{   …
    for ( i = 0 ; i < rows; i++)
        for ( j = 0; j < cols; j++)
            total += table[i][j];
    …
}
```

But you may still need it just to use the array.

Passing Variable-Sized Arrays

```
void sumTable(int rows, int cols, int table[][cols])
{
   ...
}
```

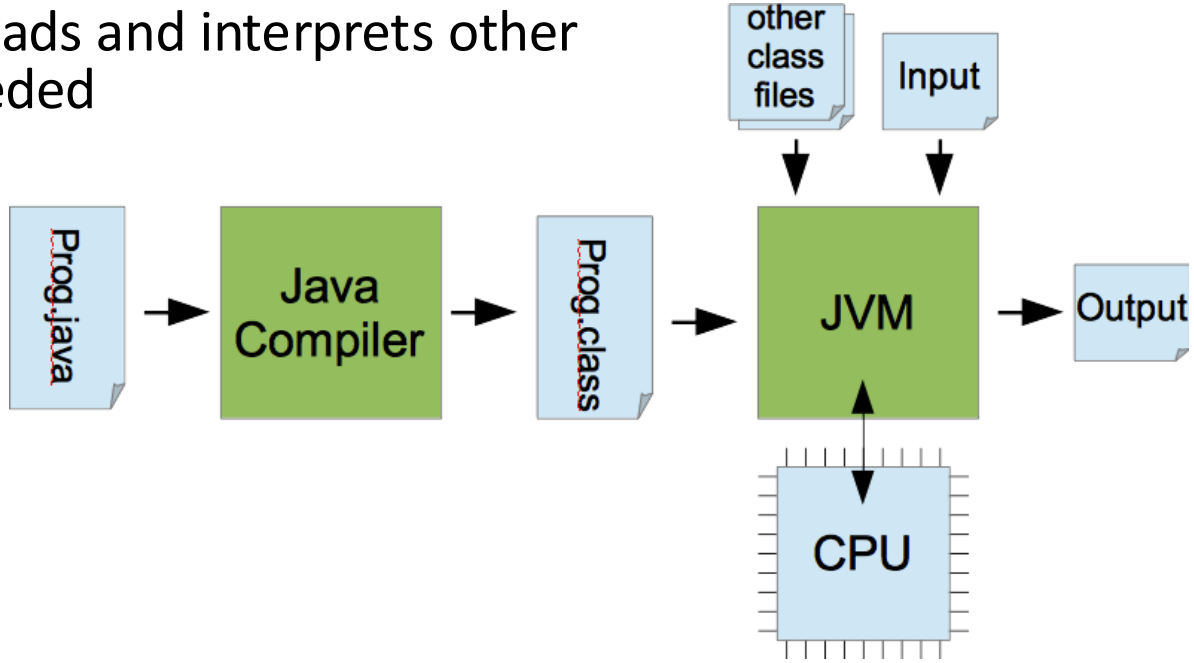
- We depend on the caller to tell us the truth.

```
int tbl[10][12];

// Good client code.
sumTable( 10, 12, tbl );
// Evil client code.
sumTable( 20, 13, tbl );
```

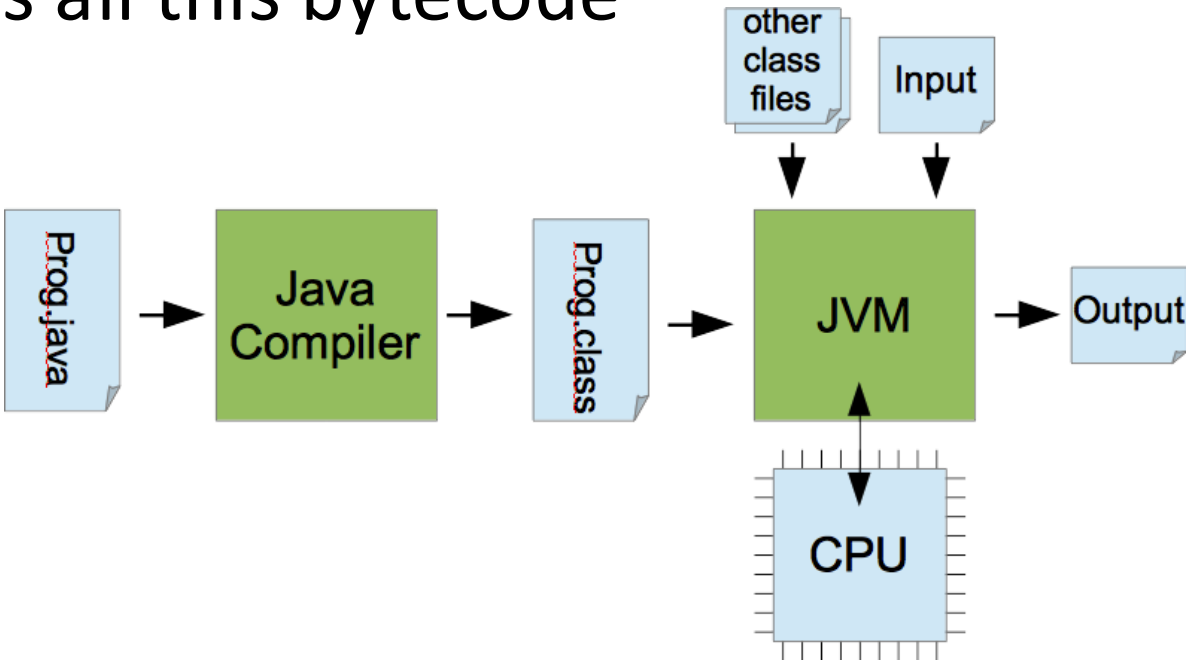
Executing Java Programs

- Java source code is *compiled* into Java class file containing *bytecode*
  - A platform-independent, intermediate representation
- To run it, we need an interpreter, the Java Virtual Machine
  - Takes a class file as input, runs native machine code to simulate each bytecode instruction
  - Automatically loads and interprets other class files as needed



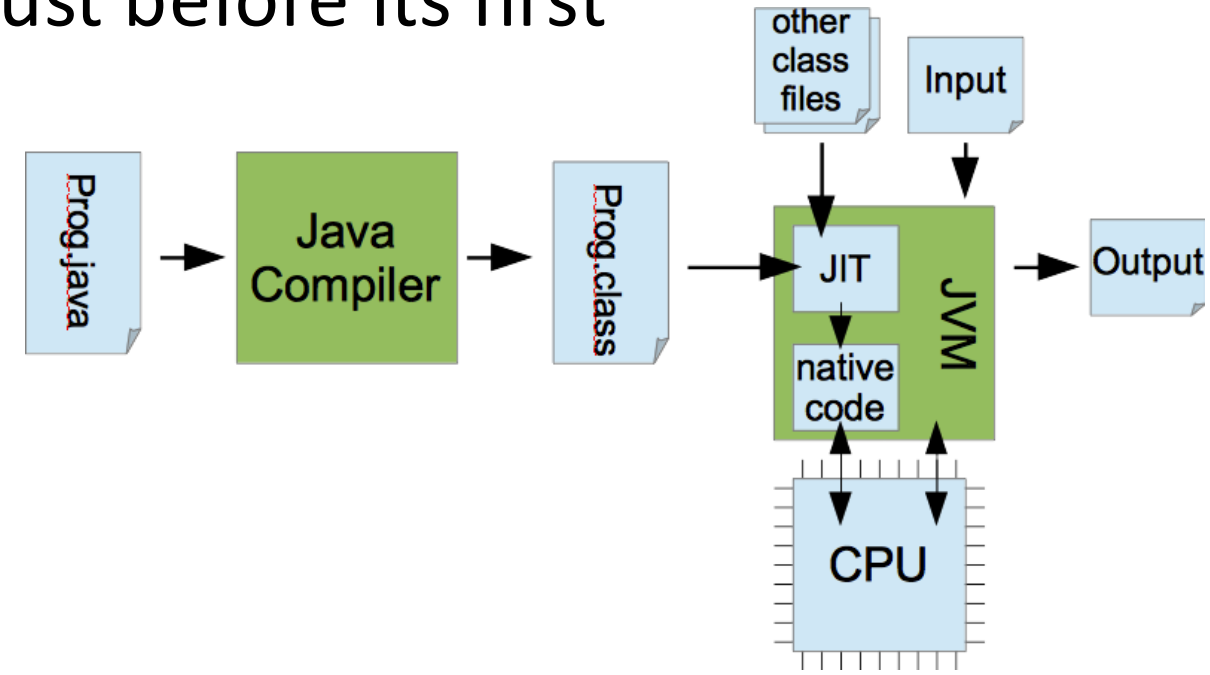
Executing Java Programs

- This is great. The class files for our compiled program are platform independent.
- But, some extra overhead is incurred as the JVM interprets all this bytecode



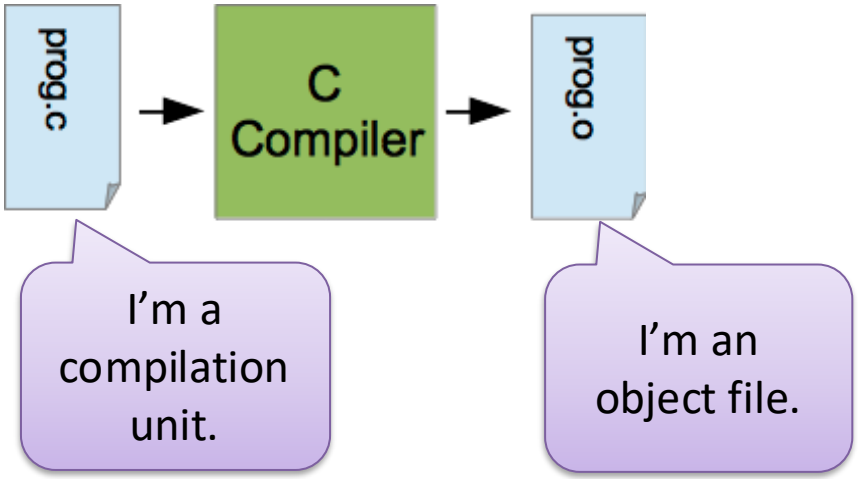
Executing Java Programs

- With Just-In-Time compilation, Java can get closer to native processor speeds
- Each method is compiled to native machine instructions just before its first execution



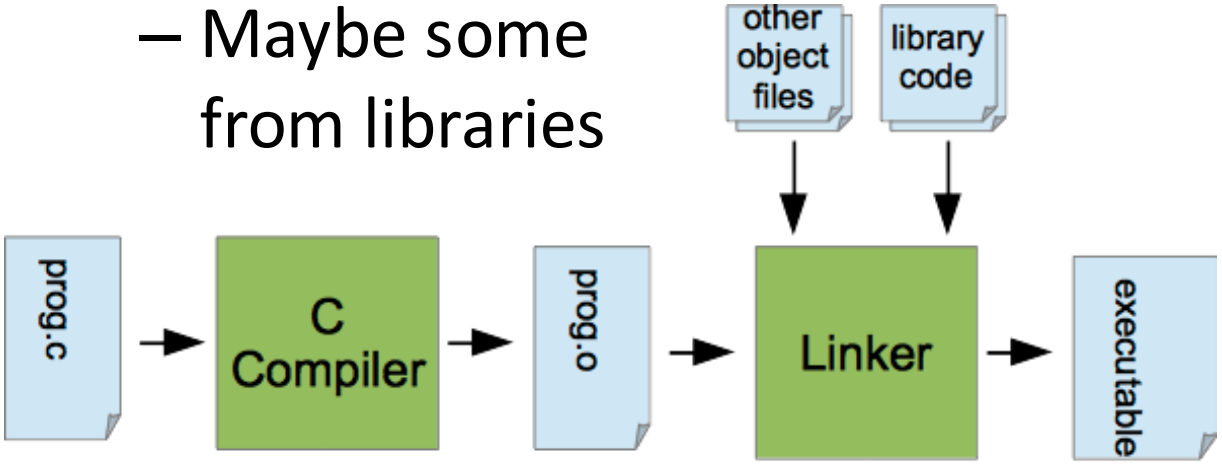
Executing C Programs

- A C Compiler generates native machine code for the target processor
- One *compilation unit* generates one *object file*



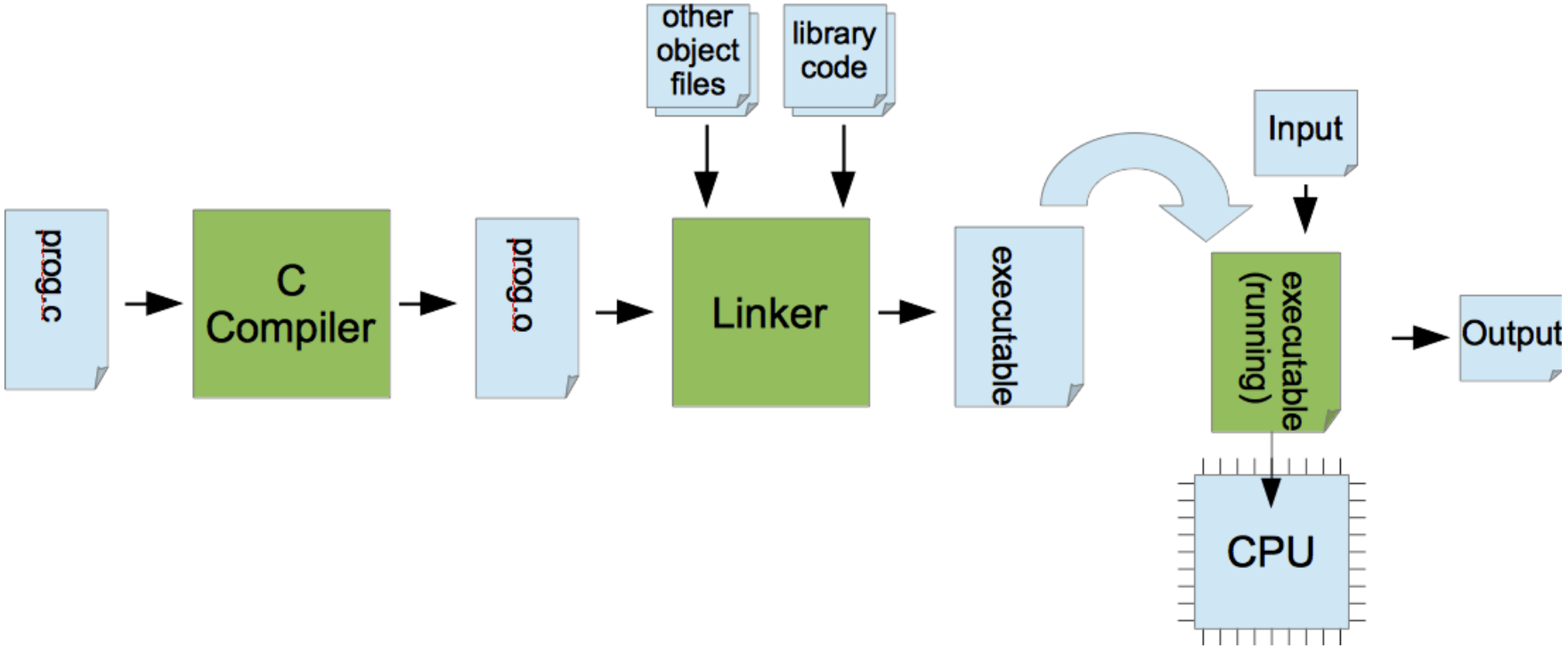
Executing C Programs

- A linker combines object files to create an executable program
  - Maybe some other objects we wrote
  - Maybe some from libraries



Executing C Programs

- The executable is ready to run
- Just load it into memory and start running at the top of main()

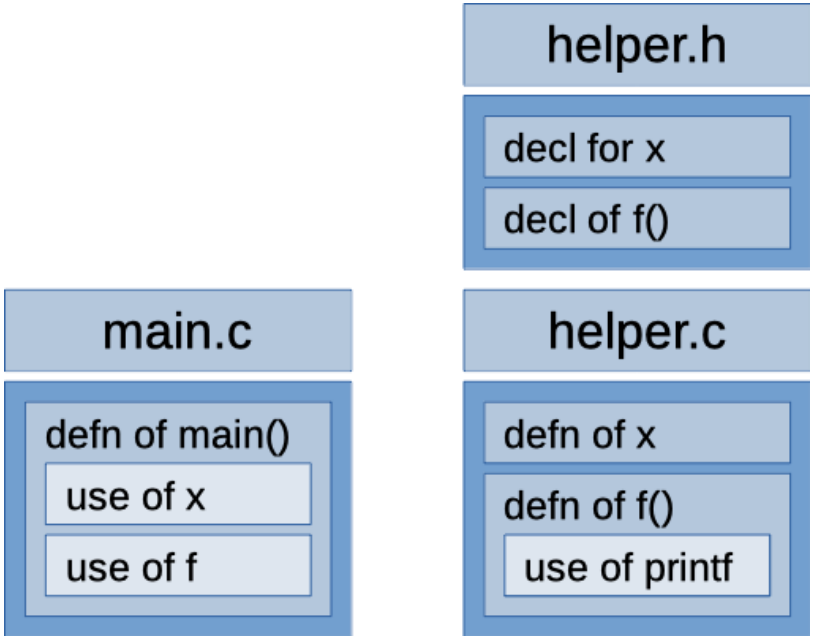


Compiled vs Interpreted

- Each approach has advantages.
  - Compiled code (C) will usually execute faster.
  - Interpreted code (Java) offers greater platform independence.
  - Interpreted code (Java) can sometimes give better support for debugging and error messages.
  - Compiling down to machine code and having all the code at compile-time can give more opportunities for code analysis and optimization.

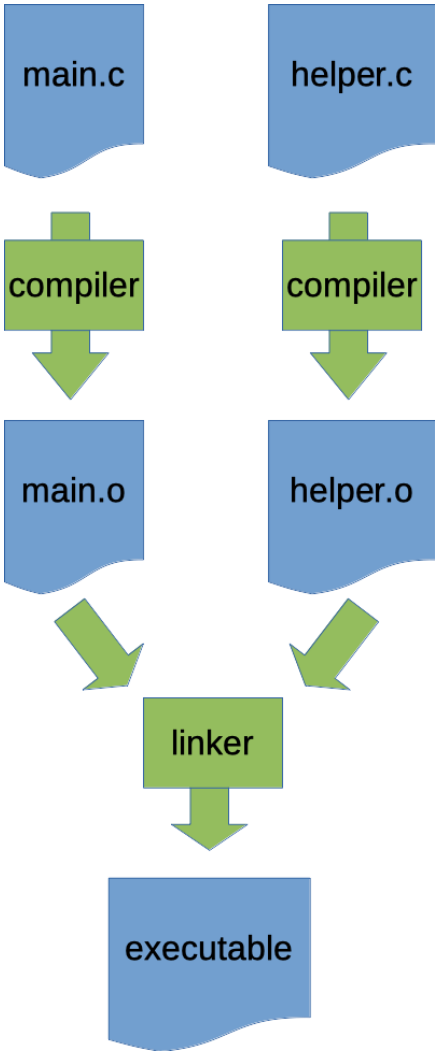
Building a Program

- Using the same example from lecture 5.
  - Helper component with:
    - Global variable x
    - Function f()
  - Main component with:
    - Function main()



Building a Program

- gcc can compile everything and link, all with one command.
- But, this is a bad way to build a large program.
  - It's more efficient to compile each object separately.
  - Then link all the objects together.
  - Makes it possible just re-compile selected parts as needed.



Separate Compile-and-Link

- The **-c** option for gcc says to just compile.
  - This generates an object file (ending in .o) as output.

```
$ gcc -Wall -std=c99 -c main.c
```

```
$ gcc -Wall -std=c99 -c helper.c
```

- Then we can ask the compiler to link these objects together.

```
$ gcc main.o helper.o -o main
```

Separate Compile-and-Link

- Some gcc options are for compile-time

```
$ gcc -Wall -std=c99 -c main.c
```

```
$ gcc -Wall -std=c99 -c helper.c
```

We're compile-time options.

Output filename is chosen based on the source file.

- We only need the -o option when we're linking.

```
$ gcc main.o helper.o -o main
```

Looking Inside an Object

- With the **nm** command, we can look inside an object or library.

```
$ nm main.o
                 U f
0000000000000000 T main
                 U x
```

I'm defined in main.o

But not us.

```
$ nm helper.o
0000000000000000 T f
                 U printf
0000000000000000 D x
```

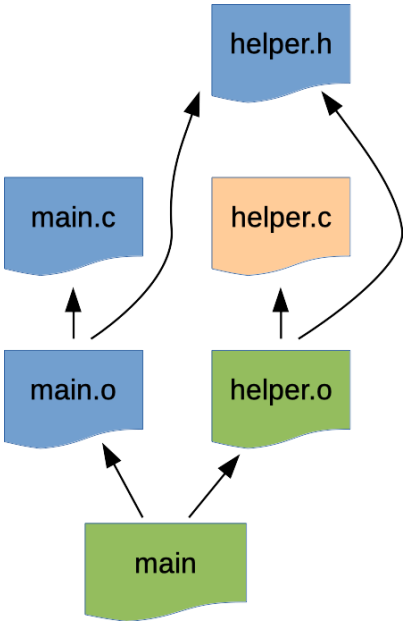
OK, the linker will hook the uses of these symbols up with their definitions.

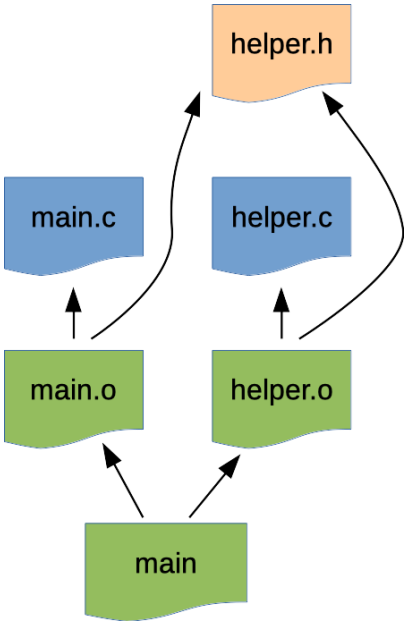
Meet Make

- Consider, our project consists of multiple files, some are source files and others are created during a build
- To build a project, we need to know:
  - the various parts of the project
  - what *dependencies* exist between files we build and the files they depend on
  - *rules* to rebuild files when a dependency is changed
- Make gives us a way to store and use this information:
  - Makefile : A text file format with dependency and build instructions
  - make : A tool for following these instructions

Thinking about Dependencies

- A project's dependencies tell us what needs to be re-built if a source file changes.





Makefile Syntax

- Makefiles consist mostly of *rules*.
  - They give a *target*, something that can be built.
  - A list of *prerequisites*, indicating what the target needs and when it must be rebuilt.
  - A list of shell commands, instructions for rebuilding the target from its dependencies.

```
target: prereq1 prereq2 ...
        shell-cmd1
        shell-cmd2
        ...
```

Command lines all start with a hard tab.

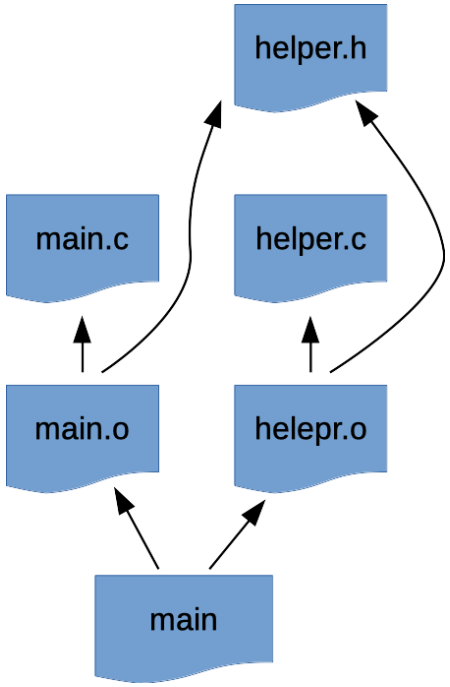
A Simple Makefile

- A rule for each target
  - Each with prerequisites
  - And a shell command to rebuild the target

```
main: main.o helper.o
    gcc main.o helper.o -o main

main.o: main.c helper.h
    gcc -Wall -std=c99 -c main.c

helper.o: helper.c helper.h
    gcc -Wall -std=c99 -c helper.c
```



Making Sense of a Makefile

To build this

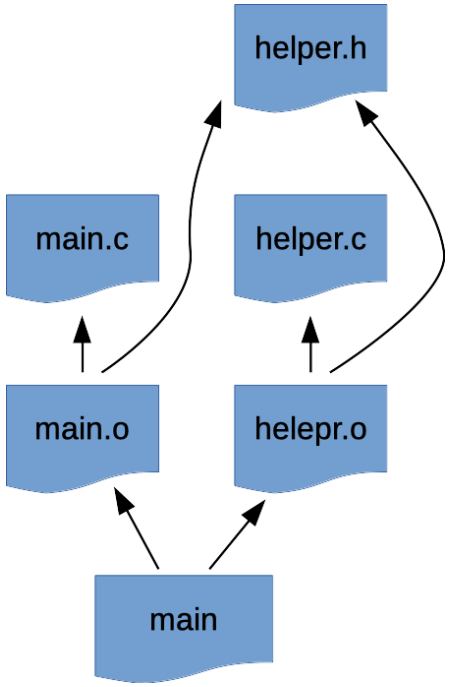
… first you must build these

… and here's how.

```
main: main.o helper.o
    gcc main.o helper.o -o main

main.o: main.c helper.h
    gcc -Wall -std=c99 -c main.c

helper.o: helper.c helper.h
    gcc -Wall -std=c99 -c helper.c
```



Making Sense of a Makefile

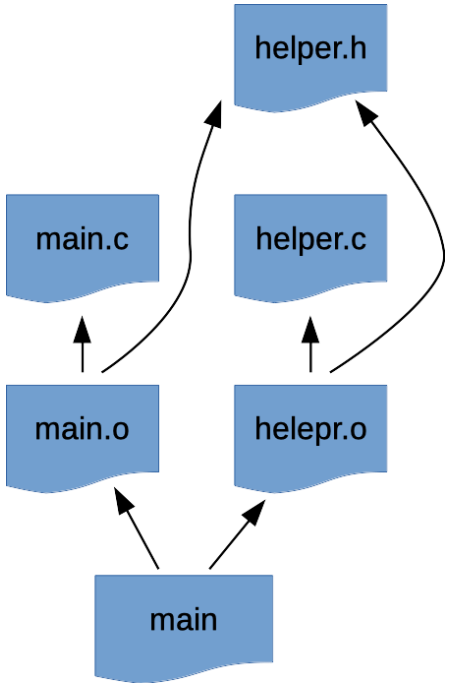
First target is the default.

You can have blank lines, and comments.

```
# This is the default target
main: main.o helper.o
    gcc main.o helper.o -o main

# Building each object file
main.o: main.c helper.h
    gcc -Wall -std=c99 -c main.c

helper.o: helper.c helper.h
    gcc -Wall -std=c99 -c helper.c
```



Using Makefiles

- You can specify the target to make.

```
$ make
gcc -Wall -std=c99 -c main.c
gcc -Wall -std=c99 -c helper.c
gcc main.o helper.o -o main
```

- make is lazy. It only rebuilds what's needed.

```
$ make
make: `main' is up to date.
```

- We can force it to rebuild.

```
$ touch main.c
$ make main.o
gcc -Wall -std=c99 -c main.c
```

This updates the modification time on a file.

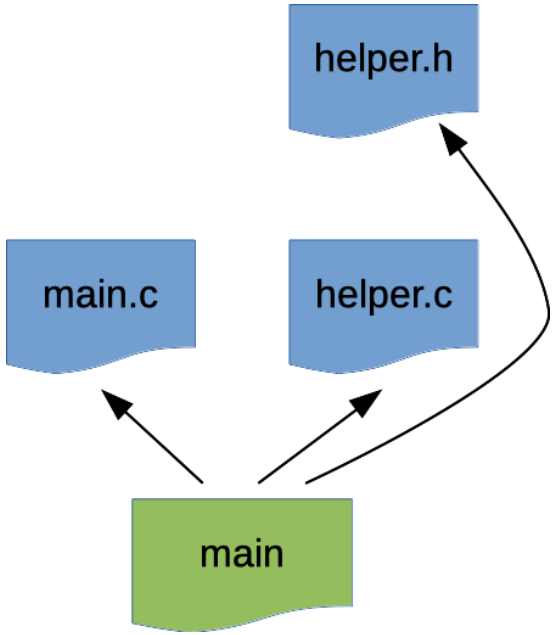
You can choose your own target.

A “Simplified” Makefile

- We can take some shortcuts, but make won’t be able to do as good a job.

```
main: main.c helper.c helper.h
    gcc -Wall -std=c99 main.c helper.c -o main
```

- This makefile will work, but it won’t work as well.



Building A Better Makefile

- Make has default rules for building common targets from sources
  - Target **`abc.o`** usually depends on **`abc.c`** with **`gcc -c`** used to build it
  - Objects can be linked to build executable targets
- Make is happy to tell you its default rules.

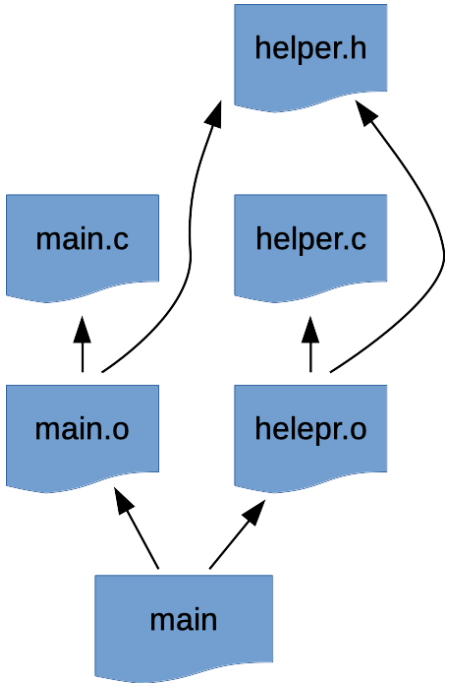
```
$ make -p -f/dev/null
```

- Using these, we can make a new makefile almost as good as the first.

```
main: main.o helper.o
main.o: main.c helper.h
helper.o: helper.c helper.h
```

No build instructions.

Default rules for **everything**.



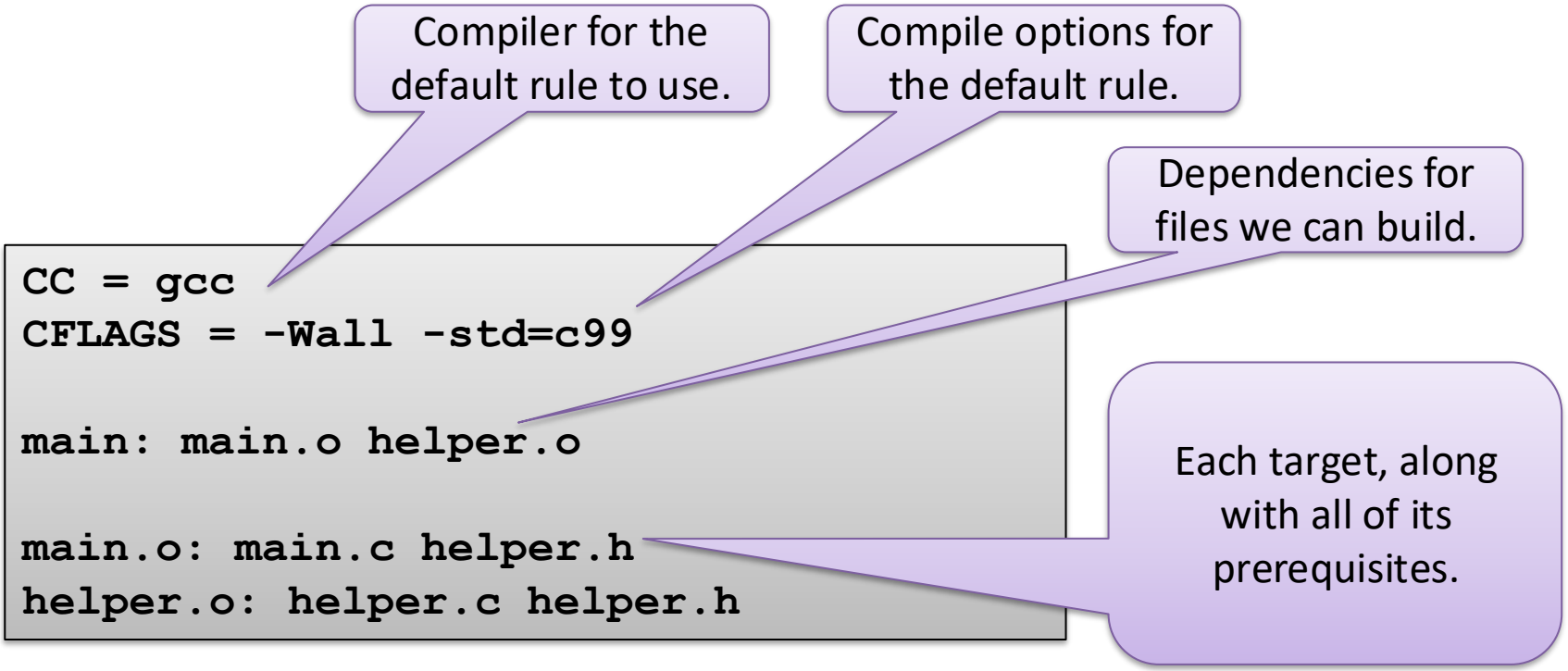
make Variables

- Make supports variables
  - To configure the default rules
  - And, whatever else you want

- The default rules use variables like:
  - CC : the default compiler to use
  - CFLAGS : the default flags to compile with
  - LDLIBS : options for the linker (libraries, e.g., -lm)

Makefiles with Variables

- Now, we can write something that looks like a decent, short makefile:



More About Make

- Make supports syntax to let you:
  - Define targets that don't build anything
    For example: make clean
  - Choose a particular makefile to use
    For example: make -f Makefile2
  - Define your own reusable rules
  - Decide what to do if a build command fails
- Oh, and I once had a job interview with Stu Feldman, the guy who created make.

On Beyond Make

- Make captures some important ideas, but it has some disadvantages
  - Build instructions in shell hurt portability
- Other tools perform the same kind of task
  - CMake : platform independent, can generate makefiles for various platforms
  - Ant : built in Java, configuration in XML
  - Maven : configuration in XML, dependency management
  - Lots more